

Efficient Batched IBE from Lattices in the Standard Model

Saisi Xiong¹ and Jie Chen²

¹ East China Normal University, Software Engineering Institute, Shanghai, China

² Wuhan University, School of Cyber Science and Engineering, Wuhan, China

Abstract. In this work, we present the first lattice-based construction of batched IBE in the standard model, whose security is proven under the succinct LWE assumption. Prior batched IBE schemes are only known either based on pairing-based assumptions or in the random oracle model. Moreover, our scheme is shown to be highly efficient, as the master public key, decryption key, and ciphertext are independent of the batch size B .

Technically, we mainly rely on an insightful observation: batched IBE can be obtained solely from Inner-Product Encryption (IPE). To satisfy the efficiency requirements of batched IBE, we require an IPE scheme that owns two important features: *decomposable* key generation and *compact* components. Finally, we show how to construct such an IPE scheme from the well-known BGG+14 IPE scheme via careful modification.

1 Introduction

Identity-Based Encryption (IBE), which was introduced by [Sha84], enables the encryption of messages using a master public key and an identity, thus removing the need for *Public Key Infrastructure* (PKI). In an IBE scheme, any user with a decryption key associated with the matched identity can decrypt and recover the message.

Motivation: Mempool Privacy and Scalability. In blockchain application scenarios, when users want to publish a transaction to a blockchain, they need to submit transactions to a public memory pool (mempool for short) in advance. Then, miners will select a batch of transactions that will be included in the next block on the blockchain. Since transaction details are publicly visible, an adversarial miner can exploit this information to rerank or selectively choose transactions for profit. The additional value that a miner can derive through such attacks is referred to as the *Miner Extractable Value* (MEV). A leading defense against MEV attacks is to encrypt all transactions within the mempool and ensure that the corresponding decryption keys are not released until the transactions are included in a specific block. Thereafter, the transaction sequence has been finalized, preventing attackers from profiting by manipulating transactions even if they know the transaction details.

We can adopt an IBE scheme to enforce the above encryption task. In particular, users can encrypt transactions using a future block ID or timestamp as the identity. This enables transactions to be submitted to the mempool before the corresponding decryption key is generated, thereby achieving “Encryption to the Future”. However, the use of IBE schemes may give rise to two scenarios:

- **All-or-Nothing Decryption.** If a user encrypts many transactions under the same identity (e.g., “Block #100”), a critical issue about mempool privacy arises. Once

E-mail: saisixiong@163.com (Saisi Xiong), s080001@e.ntu.edu.sg (Jie Chen)

the committee releases an IBE decryption key corresponding to “Block #100”, all transactions encrypted under this identity will be implicitly revealed. Given that the block capacity on the blockchain is limited, miners select only a subset of transactions for inclusion. Consequently, excluded transactions are exposed due to the public release of the decryption key. This undermines the privacy objectives of the mempool. Premature exposure of excluded transactions may render them vulnerable to MEV attacks in subsequent blocks.

- **Inefficient Independent Decryption.** Alternatively, if each transaction is encrypted under a unique identity (e.g. “Transaction #10000”), the problem of premature decryption will be avoided. However, since each block contains a batch of transactions, the committee would need to publish multiple distinct decryption keys for the transactions in the batch. Consequently, the communication and on-chain storage costs scale linearly with the batch size. Such linear overhead is prohibitive for high-throughput blockchains.

Solution: Batched IBE. To address these limitations, Agarwal et al. [AFP25] introduced a novel primitive called *Batched IBE*. In a batched IBE scheme, there is a trusted authority (or a committee) that issues a short secret key $\mathbf{sk}$ for an arbitrary batch of identities $\{\text{id}_1, \dots, \text{id}_B\}$ (e.g., the specific set of transactions included in a block) and a time label t . This key enables the decryption of any ciphertext ct associated with an identity id such that $\text{id} \in \{\text{id}_1, \dots, \text{id}_B\}$, while any ciphertext associated with the identity outside the batch reveals nothing, thereby ensuring the privacy of those excluded transactions. Moreover, the secret key is derived from a short digest associated with the identity set $\{\text{id}_1, \dots, \text{id}_B\}$, where the digest size is independent of B . Therefore, the size of the secret key is also independent of B , thereby avoiding the inefficient decryption.

Beyond mempool privacy, batched IBE also serves as a versatile tool for conditional decryption, enabling scalable solutions for applications such as fair *multi-party computation* (MPC), secure Dutch auctions, and privacy-preserving options trading.

Current Gap. Despite its strong motivation, constructing efficient batched IBE schemes with rigorous security guarantees remains challenging. Initially, Agarwal et al. [AFP25] constructed the first batched IBE scheme by utilizing a KZG polynomial commitment scheme along with an efficient special-purpose witness encryption scheme. However, the security of their scheme is only proven in the generic group model [Sho97], which is pretty weak. Very recently, Gong et al. [GWW25] presented a new batched IBE scheme without this drawback, which is based on the q -type assumption over pairings in the standard model. However, since all of these schemes are vulnerable to future quantum attacks, it is always of interest to construct batched IBE schemes based on assumptions beyond pairings, such as lattices. Compared with pairing-based cryptographic schemes, lattice-based schemes have gained broader popularity owing to the well-established post-quantum security guarantee. Therefore, Boneh et al. recently [BLT25] proposed a lattice-based batched decryption scheme, which is based on the Learning with Error (LWE) assumption. We note that, a batched IBE scheme can be obtained from this batched decryption scheme via a slight modification. However, a major drawback of this scheme is that it relies on random oracles. Therefore, in a spirit similar to that of Gong et al. [GWW25], we aim to provide a new batched IBE scheme from lattices in the standard model.

1.1 Related Work

Batch decryption was also proposed to achieve mempool privacy and shares similar definitions to batched IBE. In a batch decryption scheme, a compact decryption key can be generated using the master secret key msk and a id set $\{r_i\}_{i \in [B]}$. This key subsequently enables decryption of $\{\text{ct}_i\}_{i \in [B]}$ where ct_i was encrypted with $\text{id } r_i$. The difference lies in the fact that batch decryption only requires the decryption key to be compact, whereas

batched IBE also necessitates the time to generate the decryption key be sufficiently short. Choudhuri et al. proposed the first batch decryption scheme in [CGPP24]. In their scheme, each message is encrypted under an epoch id eid and at the beginning of each epoch, a new Setup must be executed. Subsequently, [CGPW24] improved upon this research result, constructing a scheme that only requires a one-time setup. However, these schemes allow only a single decryption key to be published for every epoch, which limits their application. Faust et al. introduced the first epoch-less batch decryption scheme called BEAT-MEV [BFOQ24]. Recently, Boneh et al. [BLT25] have proposed a lattice-based batch decryption construction which features shorter parameters.

1.2 Our Results

In this work, we propose the first lattice-based batched IBE scheme in the standard model, which is selectively secure based on the ℓ -succinct LWE assumption [Wee24]. The ℓ -succinct LWE assumption is a falsifiable assumption and is implied by the (public-coin) evasive LWE assumption and the standard LWE assumption. This assumption, as well as its various variants, has been used in recent constructions of succinct functional commitments [WW23a, WW23b, CLM23], distributed broadcast encryption [CW24, WW25], and registered attribute-based encryption [CHW25]. In this work, we assume $\ell = \text{poly}(\lambda, \log B)$, where B denotes the size of the batch set.

We summarize and compare prior batched IBE schemes in Table 1, and claim several advantages of our scheme as follows:

- **Security in the standard model.** Our scheme does not rely on any non-standard models such as the generic group model and the random oracle model. Prior to this work, only the scheme very recently proposed by Gong et al. [GWWW25] is known to be secure in the standard model, while it has to rely on a q -type assumption over pairings.
- **Compactness.** For any batched IBE schemes, it is desired to obtain a smaller master public key mpk , secret key sk , and ciphertext ct . Our result exactly achieves $|\text{mpk}| = \text{poly}(\log B)$, $|\text{sk}| = \text{poly}(\log B)$ and $|\text{ct}| = \text{poly}(\log B)$. Prior pairing-based schemes [AFP25, GWWW25] must endure a long mpk scaling with B . On the other hand, although the scheme in [BLT25] also owns these components that are comparable to our result, it relies on random oracles.

Table 1: Comparison with prior batched IBE schemes. For each scheme, we report the size of the master public key mpk , decryption key sk , ciphertext ct , and the underlying assumption. Here B stands for the batch size, “GGM” stands for the generic bilinear group model, and “ROM” stands for the random oracle model. We hide λ -related factors in the parameter size and unify the identity space into $\mathbb{Z}_q$, and it always has $q = O(B)$ and $\ell = \text{poly}(\lambda, \log B)$.

Scheme	$ \text{mpk} $	$ \text{sk} $	$ \text{ct} $	Assumption
[AFP25]	$O(B)$	$O(1)$	$O(1)$	GGM+ROM
[GWWW25]	$O(B)$	$O(1)$	$O(1)$	GGM+ROM
[GWWW25]	$O(B)$	$O(1)$	$O(1)$	q -type
[BLT25]	$O(\log^3 B)$	$O(\log B)$	$O(\log^3 B)$	LWE+ROM
This work	$O(\log^6 B)$	$O(\log^2 B)$	$O(\log^2 B)$	ℓ -succinct LWE

1.3 Technical Overview

We first consider the syntax of a batched IBE scheme. Let N be the number of users in the system. Each user is indexed by a unique identity $\text{id} \in [N]$, where $[N]$ denotes the set $\{1, \dots, N\}$. Then, a simplified batched IBE runs as follows:

- **Setup.** The setup algorithm takes the security parameter 1^λ and the maximum batch size 1^B , then outputs the master public key mpk and the master secret key msk .
- **Digest generation.** The digest-generation algorithm takes the master public key mpk , a batch set of identities $S = \{\text{id}_1, \dots, \text{id}_B\}$, then outputs a digest dig .
- **Key generation.** The key-generation algorithm takes the master secret key msk , a digest dig , and a batch label t , then outputs a secret key sk .
- **Encryption.** The encryption algorithm takes the master public key mpk , a message m , an identity id , and a batch label t' , then outputs a ciphertext ct .
- **Decryption.** The decryption algorithm takes the master public key mpk , a ciphertext ct , a secret key sk , a digest dig , a batch set of identities S , and an identity id , then outputs a message m or an empty symbol $\perp$.

For correctness, it says that the decryption algorithm will recover the message m if $\text{id} \in S$ and $t = t'$ hold simultaneously. The security requires that ct reveals nothing about m when either $\text{id} \notin S$ or $t \neq t'$. For efficiency, it critically requires that the runtime of the key generation algorithm is polylogarithmic in the batch size B . This implies that the digest dig and the secret key sk are also polylogarithmic in B .

In the following, we focus on constructing a simplified version of batched IBE, in which the labels are ignored and the decryption succeeds as long as $\text{id} \in S$. Since the difference is minor, we still call it batched IBE for simplicity of expression.

Observation: a naive relation between batched IBE and IPE. We start with an interesting observation that can connect a batched IBE scheme to a well-studied primitive called *Inner-Product Encryption* [KSW08]. An IPE scheme necessitates four algorithms of setup, key generation, encryption, and decryption. In particular, the setup algorithm takes 1^λ along with the vector length 1^ℓ , then outputs the master public key mpk_{IPE} and the master secret key msk_{IPE} . The key-generation algorithm just takes msk_{IPE} and a vector $\mathbf{y} \in \mathbb{Z}_q^{1 \times \ell}$, then outputs a secret key $\text{sk}_{\mathbf{y}}$. The encryption algorithm takes mpk_{IPE} , a message m , and a vector $\mathbf{x} \in \mathbb{Z}_q^{1 \times \ell}$, then outputs a ciphertext $\text{ct}_{\mathbf{x}}$. The decryption algorithm can employ $\text{sk}_{\mathbf{y}}$ together with $(\mathbf{x}, \mathbf{y})$ to recover m from $\text{ct}_{\mathbf{x}}$, as long as $\mathbf{x}\mathbf{y}^\top = 0$; otherwise, it outputs $\perp$ when $\mathbf{x}\mathbf{y}^\top \neq 0$.

A naive relation between batched IBE and IPE is that the former is actually a special case of the latter in terms of functionality. For a batch set $S = \{\text{id}_1, \dots, \text{id}_B\}$ and a ciphertext identity id , we can define a special polynomial $f(x) = \prod_{i=1}^B (x - \text{id}_i)$ to express the function of batched IBE:

$$\text{id} \in S \iff f(\text{id}) = 0 \text{ and } \text{id} \notin S \iff f(\text{id}) \neq 0 \quad (1)$$

A general but inefficient compiler. According to (1), we can therefore generically derive a batched IBE scheme (without satisfying the efficiency requirements) from any IPE. To this end, we can rewrite the polynomial f as the form $f(x) := \sum_{j=0}^B y_j x^j$, where y_j denotes the coefficient of f . The general compiler from IPE to batched IBE works as follows:

- **Setup.** It generates the master public key $\text{mpk} = \text{mpk}_{\text{IPE}}$ and the master secret key $\text{msk} = \text{msk}_{\text{IPE}}$, where it sets the vector length $\ell = B + 1$.

- **Digest generation.** It directly outputs $\text{dig} = S$.
- **Key generation.** It generates $\text{sk} = \text{sk}_{\mathbf{y}}$, which is associated with a vector $\mathbf{y} = (y_0, y_1, \dots, y_B)$.
- **Encryption.** It generates $\text{ct} = \text{ct}_{\mathbf{x}}$, which is associated with a vector $\mathbf{x} = (1, \text{id}, \dots, \text{id}^B)$.
- **Decryption.** It just performs the IPE decryption algorithm. Note that if $f(\text{id}) = \sum_{j=0}^B y_j \text{id}^j = \mathbf{x}\mathbf{y}^\top = 0$, the message m can be recovered properly; otherwise, it reveals nothing. This is compliant with the relation (1).

Though supporting the function of batched IBE, the above scheme does not satisfy the efficiency requirements. Indeed, constructing inefficient batched IBE schemes can be trivially achieved by relying on traditional IBE, while we stick to using IPE and will next show how to get rid of such inefficiency.

Upgrading the efficiency via decomposable and compact IPE. As noted in the first work of batched IBE [AFP25], the essential challenge of constructing batched IBE schemes is to support an efficient key generation, which should be independent of the batch size B . To address this issue, our idea is to find (lattice-based) IPE schemes with good efficiency features as well. The most desirable feature is *succinct* key generation, i.e., the runtime of key generation in an IPE scheme is independent of the vector; however, such a scheme does not exist, as the key-generation algorithm must access the entire vector $\mathbf{y}$. To get around this limitation, our solution is to find an IPE scheme with the feature of *decomposable* key generation. In more detail, it means that the key-generation algorithm can be decomposed into two sub-algorithms that deal with the vector part and the key part, respectively.

- (1) **Vector-specific sub-algorithm.** It takes the master public key mpk_{IPE} and a vector $\mathbf{y}$, then outputs an internal state st .
- (2) **Key-specific sub-algorithm.** It takes the master secret key msk_{IPE} and st , then outputs $\text{sk}_{\mathbf{y}}$ that is indistinguishable from the secret key normally generated from the key-generation algorithm in IPE.

Along this way, we can treat the vector-specific sub-algorithm as the digest-generation algorithm and the key-specific sub-algorithm as the key-generation algorithm, thus yielding a batch IBE scheme as desired! Of course, the internal state st and $\text{sk}_{\mathbf{y}}$ should be of size $\text{poly}(\log B)$ to guarantee the efficiency of our final scheme. On the other hand, despite not being clearly stated, the ciphertext of a batched IBE scheme should also be independent of the batch size B to facilitate real-world applications. Putting all these together, constructing an efficient batched IBE scheme requires at least a decomposable IPE that supports both compact secret keys and compact ciphertexts, i.e., independent of the vector length ℓ . This is quite a challenge in the study of IPE [AFV11, Xag13, BGG⁺14, KNY20], as most existing schemes just support one of the compact components. To our knowledge, known compact IPE schemes either place restrictions on inner-product vectors [WFW17] or rely on pairings [BJK15, CLR16, KLM⁺18]. On the other hand, many expressive *Attribute-Based Encryption* (ABE) schemes may already imply compact IPE, but they will not be considered in this work due to their extremely impractical efficiency.

The scheme adapted from BGG⁺14 IPE. Here, we show how to construct a concrete decomposable IPE scheme with compact parameters from lattices. Our starting point is the IPE scheme proposed by Boneh, Gentry, Gorbunov, Halevi, Nikolaenko, Segev, Vaikuntanathan, and Vinayagamurthy in EUROCRYPT 2014 [BGG⁺14], which we refer to as BGG⁺14 IPE for short. The readers familiar with BGG⁺14 may see that it actually proposed a more powerful ABE scheme for general circuits, while the efficiency of this

ABE scheme is still impractical. The IPE scheme we require is exactly a special case of the ABE scheme, so it is concerning that the impracticability reflects on the IPE scheme as well. However, we observe that the impracticability of ABE is mainly caused by the homomorphic multiplication between ciphertexts, which is unnecessary for the function of IPE. We therefore want to adapt the BGG+14 IPE scheme to a decomposable IPE scheme with compact components.

Given lattice parameters n, m, q , let $\mathbf{A} \otimes \mathbf{B}$ denote the Kronecker product between matrices $\mathbf{A}$ and $\mathbf{B}$, let $\mathbf{G}$ denote the gadget matrix [MP12] and $\mathbf{G}^{-1}$ denote the binary decomposition operator, i.e., $\mathbf{G} \cdot \mathbf{G}^{-1}(\mathbf{A}) = \mathbf{A}$ for all $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$. Then we present an overview of BGG+14 IPE over lattice:

$$\begin{aligned} \text{mpk}_{\text{IPE}} : \mathbf{A}_0 \in \mathbb{Z}_q^{n \times m}, \mathbf{A} = (\mathbf{A}_1, \dots, \mathbf{A}_\ell) \in \mathbb{Z}_q^{n \times \ell m}, \mathbf{p} \in \mathbb{Z}_q^n \\ \text{ct}_{\mathbf{x}} : \mathbf{s}\mathbf{A}_0 + \text{noise} \in \mathbb{Z}_q^{1 \times m}, \mathbf{sp} + \mathbf{m} \cdot \lfloor q/2 \rfloor + \text{noise} \in \mathbb{Z}_q, \\ \mathbf{s}(\mathbf{A} - \mathbf{x} \otimes \mathbf{G}) + \text{noise} \in \mathbb{Z}_q^{1 \times \ell m} \\ \text{sk}_{\mathbf{y}} : \mathbf{k} \in \mathbb{Z}^{2m} \text{ s.t. } [\mathbf{A}_0 | \mathbf{A}(\mathbf{y}^\top \otimes \mathbf{I}_m)] \cdot \mathbf{k} = \mathbf{p} \end{aligned}$$

where $\mathbf{s} \leftarrow \mathbb{Z}_q^{1 \times n}$ is the ciphertext randomness, and the master secret key msk_{IPE} is the trapdoor for matrix $\mathbf{A}_0$. With msk_{IPE} , it is easy to derive a low-norm vector $\mathbf{k}$ for any given matrix $\mathbf{B}$ such that $[\mathbf{A}_0 | \mathbf{B}] \cdot \mathbf{k} = \mathbf{p}$. To decrypt the ciphertext $\text{ct}_{\mathbf{x}}$, we only rely on the mechanism of homomorphic addition and scalar multiplication proposed by Boneh et al.. Concretely, it has

$$\begin{aligned} \left(\mathbf{s}(\mathbf{A} - \mathbf{x} \otimes \mathbf{G}) + \text{noise} \right) (\mathbf{y}^\top \otimes \mathbf{I}_m) &= \mathbf{s}\mathbf{A}(\mathbf{y}^\top \otimes \mathbf{I}_m) - \mathbf{s}(\mathbf{x} \otimes \mathbf{G})(\mathbf{y}^\top \otimes \mathbf{I}_m) + \text{noise} \\ &= \mathbf{s}\mathbf{A}(\mathbf{y}^\top \otimes \mathbf{I}_m) - \mathbf{xy}^\top \cdot \mathbf{s}\mathbf{G} + \text{noise} \\ &\approx \mathbf{s}\mathbf{A}(\mathbf{y}^\top \otimes \mathbf{I}_m) \end{aligned}$$

where the second $=$ follows the tensor-product property $(\mathbf{A} \otimes \mathbf{B})(\mathbf{C} \otimes \mathbf{D}) = (\mathbf{AC}) \otimes (\mathbf{BD})$. Notably, $\mathbf{s}\mathbf{A}(\mathbf{y}^\top \otimes \mathbf{I}_m)$ can be estimated if $\mathbf{xy}^\top = 0$. With $\text{sk}_{\mathbf{y}}$, it can therefore recover

$$\begin{aligned} \mathbf{sp} + \mathbf{m} \cdot \lfloor q/2 \rfloor + \text{noise} - [\mathbf{s}\mathbf{A}_0 | \mathbf{s}\mathbf{A}(\mathbf{y}^\top \otimes \mathbf{I}_m)] \cdot \mathbf{k} &= \mathbf{sp} + \mathbf{m} \cdot \lfloor q/2 \rfloor + \text{noise} - \mathbf{sp} \\ &= \mathbf{m} \cdot \lfloor q/2 \rfloor + \text{noise} \\ &\approx \mathbf{m} \cdot \lfloor q/2 \rfloor \end{aligned}$$

We show how to build a decomposable IPE with compact components from the above IPE scheme, in which we mainly analyze the efficiency of key generation and encryption:

- The key generation of the above IPE scheme is inherently decomposable. In the vector-specific sub-algorithm, it just outputs a vector commitment to $\mathbf{y}$, i.e., $\text{st} = \mathbf{A}(\mathbf{y}^\top \otimes \mathbf{I}_m) \in \mathbb{Z}_q^{n \times m}$. In the key-specific sub-algorithm, it can generate the secret key $\text{sk}_{\mathbf{y}} = \mathbf{k} \in \mathbb{Z}^{2m}$ just by taking msk and st . Since n, m and q are fixed system parameters, st and sk seem to be independent of the vector length ℓ .
- The ciphertext of the above IPE scheme is not compact, while we can compress it from $O(\ell)$ to $O(\log \ell)$ by employing the recent matrix-commitment scheme of Wee [Wee25]. Specifically, such long ciphertext is caused by the component $\mathbf{s}(\mathbf{A} - \mathbf{x} \otimes \mathbf{G})$. To address this issue, the matrix-commitment scheme of Wee shows that there is an efficient and deterministic algorithm to compute a commitment $\mathbf{C}_{\mathbf{x}} \in \mathbb{Z}_q^{n \times m}$ and a low-norm opening $\mathbf{Z} \in \mathbb{Z}_q^{\ell m}$ such that

$$\mathbf{C}_{\mathbf{x}} \cdot \mathbf{V}_\ell = \mathbf{x} \otimes \mathbf{G} - \mathbf{B} \cdot \mathbf{Z} \in \mathbb{Z}_q^{n \times \ell m} \quad (2)$$

where $\mathbf{B}$ comes from the public parameters pp_{com} (independent of the vector length ℓ) of the matrix-commitment scheme and $\mathbf{V}_\ell \in \mathbb{Z}_q^{m \times \ell m}$ is a fixed low-norm verification matrix that is publicly derived from pp_{com} and the width ℓm . Thus, in the adapted IPE scheme, it encrypts using a succinct commitment $\mathbf{C}_\mathbf{x}$, instead of $\mathbf{x} \otimes \mathbf{G}$. This yields a short ciphertext independent of the vector length ℓ . On the other hand, since the implementation of the matrix-commitment scheme requires the ℓ -succinct LWE assumption [Wee24], our final scheme will also rely on this assumption.

Above all, we present our new decomposable IPE scheme as follows:

$$\begin{aligned} \text{mpk}_{\text{IPE}} : & \text{pp}_{\text{com}}, \mathbf{B}_1 \in \mathbb{Z}_q^{n \times m}, \mathbf{p} \in \mathbb{Z}_q^n \\ \text{ct}_\mathbf{x} : & \mathbf{s}\mathbf{B} + \text{noise} \in \mathbb{Z}_q^{1 \times m}, \mathbf{sp} + \mathbf{m} \cdot \lfloor q/2 \rfloor + \text{noise} \in \mathbb{Z}_q, \\ & \mathbf{s}(\mathbf{B}_1 + \mathbf{C}_\mathbf{x}) + \text{noise} \in \mathbb{Z}_q^{1 \times m} \\ \text{sk}_\mathbf{y} : & \mathbf{k} \in \mathbb{Z}^{2m} \text{ s.t. } [\mathbf{B}] - \mathbf{B}_1 \mathbf{V}_\ell (\mathbf{y}^\top \otimes \mathbf{I}_m) \cdot \mathbf{k} = \mathbf{p} \end{aligned}$$

where $\mathbf{B} \in \mathbb{Z}_q^{n \times m}$ in pp_{com} serves as $\mathbf{A}_0$ and $-\mathbf{B}\mathbf{V}_\ell$ serves as $\mathbf{A}$ in the BGG+14 IPE scheme. The decryption can first recover $\mathbf{x} \otimes \mathbf{G}$ in the component $\mathbf{s}(\mathbf{B}_1 + \mathbf{C}_\mathbf{x}) + \text{noise}$ via the matrix commitment scheme (2), then proceed to obtain $\mathbf{m}$ as before.

Compiling into batched IBE. The final step is to compile our decomposable IPE scheme into a concrete batched IBE scheme. The compilation is natural as claimed before, and recall that we assume the identity space $[N]$, so the underlying decomposable IPE scheme should take the vectors $\mathbf{y} \in [N^B]^{B+1}$ and $\mathbf{x} \in [N^B]^{B+1}$. However, a new problem is that the lattice modulus q must be greater than N^B and the ciphertext size is therefore linear in B . This is because the noise magnitude is enlarged by a factor of $(B+1)N^B$ when multiplying $\mathbf{y}^\top \otimes \mathbf{I}_m$ with ciphertext components when decrypting. Our solution is to replace $\mathbf{y}$ with a low-norm vector $\hat{\mathbf{y}}$, while not affecting the function of our batched IBE scheme. This can be done by further breaking $\mathbf{y} \in [N^B]^{B+1}$ into a binary decomposition form. In more detail, instead of $(\mathbf{y}, \mathbf{x})$, the decomposable IPE scheme takes new vectors $\hat{\mathbf{y}}^\top = \mathbf{G}^{-1}(\mathbf{y}^\top) \in \{0, 1\}^{B(B+1)\lceil \log N \rceil}$ and $\hat{\mathbf{x}} = \mathbf{x} \otimes \mathbf{g}'$ where $\mathbf{g}' = (1, 2^1, \dots, 2^{B\lceil \log N \rceil - 1})$. It can be seen that

$$\hat{\mathbf{x}}\hat{\mathbf{y}}^\top = (\mathbf{x} \otimes \mathbf{g}') \cdot \mathbf{G}^{-1}(\mathbf{y}^\top) = \mathbf{xy}^\top = f(\text{id})$$

where the polynomial f is as defined in (1). As a result, the noise growth factor will be at most $\text{poly}(B)$, thus leading to a small modulus q that is bounded by $\text{poly}(B)$. Therefore, we can obtain an efficient batched IBE scheme from lattices, where the master public key, secret key, and ciphertext are rather compact in size $\text{poly}(\log B)$. By the way, a disadvantage of our scheme is that the encryption runtime scales with B , as it must access the entire vector $\mathbf{x}$ to generate a matrix commitment to $\mathbf{x} \otimes \mathbf{G}$. This issue can be easily resolved in practice by considering an offline/online encryption paradigm, where the (deterministic) computation of the matrix commitment can be executed in the offline phase.

Note that the above batched IBE scheme follows a simplified syntax. The original definition of batched IBE requires the secret key associated with an extra label t and the ciphertext associated with a label t' . The decryption fails if $t \neq t'$. We remark that this actually models the function of normal IBE schemes, so we decide to incorporate our simplified batched IBE with the lattice-based IBE scheme in [ABB10]. This is feasible as the underlying BGG+14 IPE scheme and the IBE scheme share the same structure. Finally, we derive the first efficient batched IBE scheme from lattices, and more details are presented in Section 3.

Threshold Extension. In a batched IBE scheme, a central authority holds the system's master private key and can generate the corresponding decryption key for an identity set. In contrast, threshold batched IBE supports multiple authorities, each possessing a

“partial” master private key and able to generate partial decryption keys for an identity set. When the number of cooperating users exceeds a predetermined threshold, they can collectively utilize their “partial” decryption keys corresponding to the same identity set to generate a complete decryption key capable of decrypting ciphertexts. To thresholdize our batched IBE scheme, we can employ the technique from [BKP13] to secret share the master secret key, which is the trapdoor of the matrix $\mathbf{B}$. While this technique requires two rounds of interaction, it aligns well with the operational model of blockchains—the dominant application context for Batched IBE. Since committees already engage in inherently interactive consensus protocols, embedding these additional rounds incurs negligible latency overhead relative to the consensus process.

2 Preliminaries

Notations. For a finite set S , we write $s \leftarrow S$ to denote that s is picked uniformly from S . Then, we use $|S|$ to denote the size of S . Let $\approx_s$ denote two distributions being statistically indistinguishable, $\approx_c$ denote two distributions being computationally indistinguishable, $\Delta(A, B)$ denote the statistical distance between the two distributions A and B . For any $x \in \{0, 1\}^n$, we use $x[w]$ to denote the w -th bit of x . For a matrix $\mathbf{A}$, we use $\tilde{\mathbf{A}}$ to denote the Gram-Schmidt orthogonalization of $\mathbf{A}$, $\|\mathbf{A}\|_2$ to denote the l_2 -norm of $\mathbf{A}$, $\|\mathbf{A}\|$ to denote the l_∞ -norm of $\mathbf{A}$. We use $\mathbb{Z}_p^*$ to denote $\mathbb{Z}_p \setminus \{0\}$ and use $\mathbf{G} = \mathbf{I}_n \otimes \mathbf{g} \in \mathbb{Z}^{n \times m}$ to denote gadget matrix where $\mathbf{g} = (1, 2^1, \dots, 2^{\lceil \log q \rceil - 1}, 0, \dots, 0) \in \mathbb{Z}^{1 \times m}$.

Tensor Product (Kronecker Product). The tensor product is a binary operation acting on two vector spaces that maps them into a new vector space while preserving bilinearity. The tensor product for matrices $\mathbf{A} = (a_{i,j}) \in \mathbb{Z}^{n \times m}$, $\mathbf{B} \in \mathbb{Z}^{\ell \times \lambda}$ is defined as:

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{1,1}\mathbf{B} & \cdots & a_{1,m}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{n,1}\mathbf{B} & \cdots & a_{n,m}\mathbf{B} \end{bmatrix} \in \mathbb{Z}^{n\ell \times m\lambda}$$

We have the following properties:

$$(\mathbf{A} \otimes \mathbf{B})(\mathbf{C} \otimes \mathbf{D}) = (\mathbf{AC}) \otimes (\mathbf{BD})$$

2.1 Lattice Background

Lemma 1 (Lattice Trapdoor [BGG⁺14]). *Let λ be a security parameter and let q, ℓ, m be lattice parameters. There are efficient algorithms with the properties below:*

TrapGen $(1^n, 1^m, q) \rightarrow (\mathbf{A}, \mathbf{T}_{\mathbf{A}})$. *A randomized algorithm that receives parameters $1^n, 1^m, q$ where $m = \Theta(n \log q)$. It then outputs a full-rank matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a basis $\mathbf{T}_{\mathbf{A}} \in \mathbb{Z}^{m \times m}$ for $\Lambda_q^\perp(\mathbf{A})$ such that $\mathbf{A}$ is $\text{negl}(n)$ -close to uniform and $\|\tilde{\mathbf{T}}_{\mathbf{A}}\| \leq O(\sqrt{n \log q})$, with all but negligible probability in n .*

SamplePre $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \mathbf{B}, \sigma) \rightarrow \mathbf{R}$. *A randomized algorithm that receives a matrix $\mathbf{A}$, its trapdoor $\mathbf{T}_{\mathbf{A}}$, a target matrix $\mathbf{B}$ and a gaussian parameter σ that satisfies $\sigma \geq \|\tilde{\mathbf{T}}_{\mathbf{A}}\| \omega(\sqrt{\log m})$. It then outputs a matrix $\mathbf{R}$ distributed statistically close to $\mathcal{D}_{\Lambda_q^\perp(\mathbf{A}), \sigma}$.*

ExtendRight $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \mathbf{B}) \rightarrow \mathbf{T}_{[\mathbf{A}|\mathbf{B}]}$. *A deterministic algorithm that receives full-rank matrices $\mathbf{A}, \mathbf{B} \in \mathbb{Z}_q^{n \times m}$, a basis $\mathbf{T}_{\mathbf{A}}$ of $\Lambda_q^\perp(\mathbf{A})$ and a gaussian parameter $\sigma \geq \|\tilde{\mathbf{T}}_{\mathbf{A}}\| \omega(\log n)$. It then outputs a matrix $\mathbf{T}_{[\mathbf{A}|\mathbf{B}]}$ distributed statistically close to $(\mathcal{D}_{\Lambda_q^\perp([\mathbf{A}|\mathbf{B}]), \sigma})^{2m}$ such that $\|\tilde{\mathbf{T}}_{[\mathbf{A}|\mathbf{B}]}\| = \|\tilde{\mathbf{T}}_{\mathbf{A}}\|$.*

$\text{ExtendLeft}(\mathbf{A}, \mathbf{B}, \mathbf{T}_\mathbf{B}, \mathbf{R}) \rightarrow \mathbf{T}_{[\mathbf{A}|\mathbf{AR}+\mathbf{B}]}$. A deterministic algorithm that receives full-rank matrices $\mathbf{A}, \mathbf{B}, \mathbf{R} \in \mathbb{Z}_q^{n \times m}$, a basis $\mathbf{T}_\mathbf{B}$ of $\Lambda_q^\perp(\mathbf{B})$ and a gaussian parameter $\sigma \geq \|\tilde{\mathbf{T}}_\mathbf{A}\| \omega(\sqrt{\log n})$. It then outputs a matrix $\mathbf{T}_{[\mathbf{A}|\mathbf{AR}+\mathbf{B}]}$ distributed statistically close to $(\mathcal{D}_{\Lambda_q^\perp([\mathbf{A}|\mathbf{AR}+\mathbf{B}])}, \sigma)^{2m}$ such that $\|\tilde{\mathbf{T}}_{[\mathbf{A}|\mathbf{AR}+\mathbf{B}]}\| = \|\tilde{\mathbf{T}}_\mathbf{B}\|(1 + \|\mathbf{R}\|_2)$.

Lemma 2 (Matrix Commitment [Wee25]). Let λ be a security parameter and let q, ℓ, m be lattice parameters. For $\text{pp} := (\mathbf{B}, \mathbf{W}, \mathbf{T})$, where $\mathbf{B} \in \mathbb{Z}_q^{n \times m}$, $\mathbf{W} \in \mathbb{Z}_q^{2m^2 n \times m}$, $\mathbf{T} \in \mathbb{Z}_q^{(2m^2+1)m \times 2m^3}$, $[\mathbf{I}_{2m^2} \otimes \mathbf{B}|\mathbf{W}]\mathbf{T} = \mathbf{I}_{2m^2} \otimes \mathbf{G}$, there exists follow efficient algorithms $(\text{Com}, \text{Ver}, \text{Open})$ where

$\text{Com}(\text{pp}, \mathbf{x}) \rightarrow \mathbf{C}_\mathbf{x}$. It receives the public parameter pp and a vector $\mathbf{x} \in \mathbb{Z}_q^\ell$ and outputs the commitment $\mathbf{C}_\mathbf{x} \in \mathbb{Z}_q^{n \times m}$.

$\text{Ver}(\text{pp}, 1^\ell) \rightarrow \mathbf{V}$. It receives the public parameter pp and the vector length ℓ and outputs a verification matrix $\mathbf{V} \in \mathbb{Z}_q^{m \times \ell m}$.

$\text{Open}(\text{pp}, \mathbf{x}) \rightarrow \mathbf{Z}_\mathbf{x}$. It receives the public parameter pp and the vector $\mathbf{x} \in \mathbb{Z}_q^\ell$ and outputs an opening $\mathbf{Z}_\mathbf{x} \in \mathbb{Z}_q^{m \times \ell m}$.

For all $\ell \in \mathbb{N}$, $\mathbf{x} \in \mathbb{Z}_q^\ell$, the matrices $\mathbf{C}_\mathbf{x} \leftarrow \text{Com}(\text{pp}, \mathbf{x})$, $\mathbf{V} \leftarrow \text{Ver}(\text{pp}, 1^\ell)$, $\mathbf{Z}_\mathbf{x} \leftarrow \text{Open}(\text{pp}, \mathbf{x})$ satisfy:

$$\begin{aligned} \mathbf{C}_\mathbf{x} \cdot \mathbf{V} &= \mathbf{x} \otimes \mathbf{G} - \mathbf{B} \cdot \mathbf{Z}_\mathbf{x} \\ \|\mathbf{V}\| &\leq O(\|\mathbf{T}\| \cdot m^4 \log q) \\ \|\mathbf{Z}_\mathbf{x}\| &\leq O(\|\mathbf{T}\| \cdot \log \ell \cdot m^7 \log q) \end{aligned}$$

Lemma 3 (Gaussian Tail Bound). Let $\mathcal{D}_{\mathbb{Z}, \sigma}$ denote the discrete Gaussian distribution over $\mathbb{Z}$ with parameter $\sigma > 0$. For any $\lambda \in \mathbb{N}$, we have

$$\Pr[\|\mathbf{x}\| \geq \sqrt{\lambda} \sigma \mid \mathbf{x} \leftarrow \mathcal{D}_{\mathbb{Z}, \sigma}] \leq 2^{-\lambda}$$

Lemma 4 (Left-over Hash Lemma). Let λ be a security parameter and let q, ℓ, m be lattice parameters. Let P be a probability distribution over $\mathbb{Z}^m$ where $H_\infty(P) \geq 2n \log q$. Then we have:

$$(\mathbf{A}, \mathbf{Ar} \pmod{q}) \approx_s (\mathbf{A}, \mathbf{u})$$

where $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}$, $\mathbf{r} \leftarrow P$, $\mathbf{u} \leftarrow \mathbb{Z}_q^n$ and $H_\infty(P)$ refers to the min-entropy of P .

Lemma 5 (Noise Rerandomization [KY16]). Let λ be a security parameter and let q, ℓ, m be lattice parameters and $r > \max\{\omega(\sqrt{\log m}), \omega(\sqrt{\log \ell})\}$. There exists an efficient algorithm ReRand that

$$(\mathbf{A}, \mathbf{x} := \mathbf{v} + \mathbf{e}, \text{ReRand}(\mathbf{A}, \mathbf{x}, \sigma)) \approx_s (\mathbf{A}, \mathbf{x}, \mathbf{y} := \mathbf{v}\mathbf{A} + \mathbf{e}')$$

where $\mathbf{v} \in \mathbb{Z}_q^{1 \times m}$, $\mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z}^{1 \times m}, r}$, $\mathbf{A} \in \mathbb{Z}^{m \times \ell}$ and $\sigma > \|\mathbf{A}\|$, $\mathbf{e}' \leftarrow \mathcal{D}_{\mathbb{Z}^{1 \times \ell}, 2r\sigma}$.

Assumption 1 (ℓ -succinct LWE [Wee24]). Let λ be a security parameter and let m, q, σ be lattice parameters, where $m \geq 2n \log q$. Then we have

$$(\mathbf{B}, \mathbf{sB} + \mathbf{e}, \mathbf{W}, \mathbf{T}) \approx_c (\mathbf{B}, \mathbf{u}, \mathbf{W}, \mathbf{T})$$

where $\mathbf{B} \leftarrow \mathbb{Z}_q^{n \times m}$, $\mathbf{s} \leftarrow \mathbb{Z}_q^{1 \times n}$, $\mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z}^{1 \times m}, \sigma}$, $\mathbf{u} \leftarrow \mathbb{Z}_q^{1 \times m}$, $\mathbf{W} \leftarrow \mathbb{Z}_q^{\ell n \times m}$, $\mathbf{T} \leftarrow [\mathbf{I}_\ell \otimes \mathbf{B}|\mathbf{W}]^{-1}(\mathbf{I}_\ell \otimes \mathbf{G})$.

2.2 Batched Identity Based Encryption

A batched IBE scheme consists of following five algorithms:

Setup($1^\lambda, 1^B$) $\rightarrow$ (mpk, msk). The setup algorithm receives the security parameter 1^λ and the batch size 1^B . It outputs the master public key mpk and the master secret key msk.

Digest(mpk, $\{id_1, \dots, id_B\}$) $\rightarrow$ dig. The digest-generation algorithm receives master public key mpk and a list of identities $id_1, \dots, id_B$, then it returns a digest dig.

Gen(msk, dig, t) $\rightarrow$ sk. The key-generation algorithm receives the master secret key msk, a digest dig, and a batch label t , then it returns a secret key sk.

Enc(mpk, m, id, t) $\rightarrow$ ct. The encryption algorithm receives the master public key mpk, a message m, an identity id and a batch label t , then it returns a ciphertext ct.

Dec(mpk, ct, sk, dig, $\{id_1, \dots, id_B\}, id, t$) $\rightarrow$ m. The decryption algorithm receives the master public key mpk, a ciphertext ct, a secret key sk, a digest dig, a list of identities $id_1, \dots, id_B$, an identity id and a batch label t , then it returns a message m.

Correctness. For all m, we require the following probability is $1 - \text{negl}(\lambda)$, where $|S| = B$ and $id \in S$.

$$\Pr \left[\text{Dec}(\text{mpk}, \text{ct}, \text{sk}, \text{dig}, S, id, t) = m \mid \begin{array}{l} (\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, 1^B) \\ \text{ct} \leftarrow \text{Enc}(\text{mpk}, m, id, t) \\ \text{dig} \leftarrow \text{Digest}(\text{mpk}, S) \\ \text{sk} \leftarrow \text{Gen}(\text{msk}, \text{dig}, t) \end{array} \right]$$

Succinctness. We require the runtime of Gen to be at most $\text{poly}(\log B)$ where B stands for the batch size. This also implies $|\text{dig}| = \text{poly}(\log B)$ and $|\text{sk}| = \text{poly}(\log B)$.

Security. The adaptive security is defined by the following game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$:

Init: The adversary $\mathcal{A}$ submits a target identity id^* and a batch label t^* .

Setup: The challenger $\mathcal{C}$ runs the setup algorithm to generate master public key mpk and master secret key msk, then sends mpk to adversary $\mathcal{A}$.

Query Phase 1: The adversary $\mathcal{A}$ may adaptively make secret key queries:

- $\mathcal{A}$ sends a list S of B identities along with a batch label t to $\mathcal{C}$. If $id^* \in S$ and $t = t^*$, then $\mathcal{A}$ loses the game.
- Otherwise, $\mathcal{C}$ computes $\text{dig} \leftarrow \text{Digest}(\text{mpk}, S)$ and $\text{sk} \leftarrow \text{Gen}(\text{msk}, \text{dig}, t)$, then sends sk to $\mathcal{A}$.

Challenge: The adversary $\mathcal{A}$ submits a pair of challenge messages (m_0, m_1) . The challenger $\mathcal{C}$ picks a random challenge bit $b \in \{0, 1\}$ and replies with $\text{ct} \leftarrow \text{Enc}(\text{mpk}, m_b, id^*, t^*)$.

Query Phase 2: $\mathcal{A}$ may continue to request a polynomial number of queries like those in Query Phase 1, with the same restriction that $\mathcal{A}$ cannot request a secret key for a list S and batch label t where $id^* \in S$ and $t = t^*$.

Guess: The adversary $\mathcal{A}$ outputs a bit b' and wins if $b' = b$.

We say a batched IBE scheme is selective security if for all efficient adversary $\mathcal{A}$, the advantage $\text{Adv}_{\mathcal{A}}^{\text{BIBE}}(\lambda) := |\Pr[\mathcal{A} \text{ wins}] - \frac{1}{2}| \leq \text{negl}(\lambda)$.

3 Batched IBE from succinct LWE

We set identity space $\mathcal{ID} = \mathbb{Z}_q$, $\mathbf{g}' = (1, 2^1, \dots, 2^{\lceil \log q \rceil - 1}) \in \mathbb{Z}^{1 \times \lceil \log q \rceil}$ and define an algorithm GenCoeff, which takes as input a set of identities $S = \{id_1 \in \mathbb{Z}_q, \dots\}$ and outputs a coefficient vector.

GenCoeff(S) $\rightarrow \mathbf{y}$: Suppose $|S| = B$, the algorithm first compute polynomial $P_S(X) := \prod_{\text{id} \in S} (X - \text{id}) = \sum_{i=0}^B y_i X^i$. Then, it set $\mathbf{y}' = (y_0, y_1, \dots, y_B) \in \mathbb{Z}_q^{1 \times (B+1)}$ proceeds to compute and output the binary decomposition of $\mathbf{y}'$: $\mathbf{y} := (\mathbf{G}^{-1}((\mathbf{y}')^\top))^\top \in \{0, 1\}^{1 \times (B+1) \lceil \log q \rceil}$.

3.1 Construction

Setup($1^\lambda, 1^B$): Initialize the lattice parameters n, m, q . Sample

$$\begin{aligned} (\mathbf{B}, \mathbf{T}_\mathbf{B}) &\leftarrow \text{TrapGen}(1^n, 1^m, q), \mathbf{W} \leftarrow \mathbb{Z}_q^{2m^2 n \times m}, \\ \mathbf{T} &\leftarrow \text{SamplePre}([\mathbf{I}_{2m^2} \otimes \mathbf{B} \mid \mathbf{W}], \mathbf{I}_{2m^2} \otimes \mathbf{T}_\mathbf{B}, \mathbf{I}_{2m^2} \otimes \mathbf{G}, \sigma_0) \\ \mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3 &\leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{P} \leftarrow \mathbb{Z}_q^{n \times \lambda} \end{aligned}$$

Output

$$\begin{aligned} \text{mpk} &:= (\overbrace{\mathbf{B}, \mathbf{W}, \mathbf{T}}^{\text{:=pp}}, \mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3, \mathbf{P}) \\ \text{msk} &:= (\mathbf{T}_\mathbf{B}) \end{aligned}$$

Digest(mpk, S): Compute $\mathbf{y} \leftarrow \text{GenCoeff}(S)$ and $\mathbf{V} \leftarrow \text{Ver}(\text{pp}, 1^{k(B+1)})$ where $k = \lceil \log q \rceil$.

Output

$$\text{dig}_S := -\mathbf{B}_1 \mathbf{V} (\mathbf{y}^\top \otimes \mathbf{I}_m) \in \mathbb{Z}_q^{n \times m}$$

Gen($\text{msk}, \text{dig}_S, t$): Compute $\mathbf{T}_{[\mathbf{B}|\text{dig}_S|\mathbf{B}_2+t\mathbf{B}_3]} \leftarrow \text{ExtendRight}(\mathbf{B}, \mathbf{T}_\mathbf{B}, [\text{dig}_S|\mathbf{B}_2+t\mathbf{B}_3])$

$$\text{sk} \leftarrow \text{SamplePre}([\mathbf{B}|\text{dig}_S|\mathbf{B}_2+t\mathbf{B}_3], \mathbf{T}_{[\mathbf{B}|\text{dig}_S|\mathbf{B}_2+t\mathbf{B}_3]}, \mathbf{P}, \sigma_1)$$

Enc($\text{mpk}, \text{id}, t, m$): Sample

$$\mathbf{s} \leftarrow \mathbb{Z}_q^{1 \times n}, \mathbf{e}_0 \leftarrow \mathcal{D}_{\mathbb{Z}^{1 \times m}, \chi}, \mathbf{e}_1, \mathbf{e}_2 \leftarrow \mathcal{D}_{\mathbb{Z}^{1 \times m}, \chi'}, \mathbf{e}_3 \leftarrow \mathcal{D}_{\mathbb{Z}^{1 \times \lambda}, \chi'}$$

Set $\text{ID} := (1, \text{id}, \text{id}^2, \dots, \text{id}^B) \otimes \mathbf{g}' \in \mathbb{Z}_q^{1 \times k(B+1)}$ where $k = \lceil \log q \rceil$. Compute $\mathbf{C}_{\text{ID}} \leftarrow \text{Com}(\text{pp}, \text{ID})$. Output

$$\text{ct} := \begin{pmatrix} \mathbf{c}_0 := \mathbf{s}\mathbf{B} + \mathbf{e}_0 \in \mathbb{Z}_q^{1 \times m}, & \mathbf{c}_1 := \mathbf{s}(\mathbf{B}_1 + \mathbf{C}_{\text{ID}}) + \mathbf{e}_1 \in \mathbb{Z}_q^{1 \times m} \\ \mathbf{c}_2 := \mathbf{s}(\mathbf{B}_2 + t\mathbf{B}_3) + \mathbf{e}_2 \in \mathbb{Z}_q^{1 \times m}, & \mathbf{c}_3 := \mathbf{s}\mathbf{P} + \mathbf{e}_3 + \mathbf{m} \lfloor \frac{q}{2} \rfloor \in \mathbb{Z}_q^{1 \times \lambda} \end{pmatrix}$$

Dec($\text{mpk}, \text{sk}, S, \text{id}, t, \text{ct}$): Set $\text{ID} := (1, \text{id}^1, \dots, \text{id}^B) \otimes \mathbf{g}'$ where $k = \lceil \log q \rceil$. Compute $\mathbf{y} \leftarrow \text{GenCoeff}(S)$, $\mathbf{V} \leftarrow \text{Ver}(\text{pp}, 1^{k(B+1)})$, $\mathbf{Z}_{\text{ID}} \leftarrow \text{Open}(\text{pp}, \text{ID})$ and

$$\mathbf{c}_4 = [\mathbf{c}_0|\mathbf{c}_1] \begin{pmatrix} -\mathbf{Z}_{\text{ID}} \\ -\mathbf{V} \end{pmatrix} \cdot (\mathbf{y}^\top \otimes \mathbf{I}_m)$$

Output

$$\lfloor \frac{2}{q} \cdot (\mathbf{c}_3 - [\mathbf{c}_0|\mathbf{c}_4|\mathbf{c}_2] \cdot \text{sk} \mod q) \rfloor$$

Correctness. If $\text{id} \in S$, then we have $\text{ID} \cdot \mathbf{y}^\top = P_S(\text{id}) = 0$, so

$$\begin{aligned} \mathbf{c}_4 &= [\mathbf{c}_0|\mathbf{c}_1] \begin{pmatrix} -\mathbf{Z}_{\text{ID}} \\ -\mathbf{V} \end{pmatrix} \cdot (\mathbf{y}^\top \otimes \mathbf{I}_m) \\ &= \mathbf{s}[\mathbf{B}|\mathbf{B}_1 + \mathbf{C}_{\text{ID}}] \begin{pmatrix} -\mathbf{Z}_{\text{ID}} \\ -\mathbf{V} \end{pmatrix} \cdot (\mathbf{y}^\top \otimes \mathbf{I}_m) + \text{noise}' \\ &= \mathbf{s}(-\mathbf{B}_1 \mathbf{V} - \text{ID} \otimes \mathbf{G}) \cdot (\mathbf{y}^\top \otimes \mathbf{I}_m) + \text{noise}' \\ &= \mathbf{s} \cdot \text{dig}_S - \mathbf{s}(\text{ID} \cdot \mathbf{y}^\top) \otimes \mathbf{G} + \text{noise}' \\ &= \mathbf{s} \cdot \text{dig}_S + \text{noise}' \end{aligned}$$

Then

$$\mathbf{c}_3 - [\mathbf{c}_0|\mathbf{c}_4|\mathbf{c}_2] \cdot \mathbf{sk} = \mathbf{m} \lfloor \frac{q}{2} \rfloor + \mathbf{sP} - \mathbf{s}[\mathbf{B}|\text{dig}_S|\mathbf{B}_2 + t\mathbf{B}_3] \cdot \mathbf{sk} + \text{noise} \approx \mathbf{m} \lfloor \frac{q}{2} \rfloor$$

The correctness follows from that

$$\begin{aligned} \|\text{noise}\| &= \|\mathbf{e}_3 + [\mathbf{e}_0|\text{noise}'] \cdot \mathbf{sk}\| \\ &\leq \|\mathbf{e}_3\| + \|\mathbf{sk}\| \cdot \left(\|\mathbf{e}_0\| + \left\| [\mathbf{e}_0|\mathbf{e}_1] \begin{pmatrix} -\mathbf{Z}_{\text{ID}} \\ -\mathbf{V} \end{pmatrix} \cdot (\mathbf{y}^\top \otimes \mathbf{I}_m) \right\| + \|\mathbf{e}_2\| \right) \\ &\leq (\chi + \chi')\sigma_1 \cdot O(\|\mathbf{T}\| \cdot m^8 B \log B \log^2 q) \leq \frac{q}{4} \end{aligned}$$

Parameters. We set LWE parameters $m = n \cdot \text{poly}(\lambda)$, $q = \text{poly}(n, B, \lambda)$, $\chi = \text{poly}(n, \lambda)$ to satisfy the following conditions:

$$\begin{aligned} \frac{q}{4} &> (\chi + \chi') \cdot \sigma_0 \cdot \sigma_1 \cdot B \log B \log^2 q \cdot \text{poly}(m, \lambda) && (\text{correctness}) \\ m &> 2(n+1) \log q && (\text{LHL}) \\ \sigma_0 &= \text{poly}(m, \lambda) && (2m^2\text{-succinct LWE}) \\ \sigma_1 &> \sigma_0 B \log B \log^2 q \cdot \text{poly}(m, \lambda) && (\text{Trapdoor Sampling}) \\ \chi &> \omega(\sqrt{m}) && (\text{ReRand}) \\ \chi' &> 2m\chi && (\text{ReRand}) \end{aligned}$$

3.2 Security

Theorem 1. *Let λ be the security parameter, and the lattice parameters described in Section 3.1. Then, under $2m^2$ -succinct LWE assumption, the above batched IBE construction is selectively secure.*

Proof. We prove the theorem via a sequence of games.

- **Game_{real}**: The real game. We have $\text{mpk} = \{\mathbf{B}, \mathbf{W}, \mathbf{T}, \mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3, \mathbf{P}\}$, where:

$$\mathbf{W} \leftarrow \mathbb{Z}_q^{2m^2 n \times m}, \mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3 \leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{P} \leftarrow \mathbb{Z}_q^{n \times \lambda}$$

and

$$\begin{aligned} (\mathbf{B}, \mathbf{T}_\mathbf{B}) &\leftarrow \text{TrapGen}(1^n, 1^m, q) \\ \mathbf{T} &\leftarrow \text{SamplePre}([\mathbf{I}_{2m^2} \otimes \mathbf{B}|\mathbf{W}], \mathbf{I}_{2m^2} \otimes \mathbf{T}_\mathbf{B}, \mathbf{I}_{2m^2} \otimes \mathbf{G}, \sigma_0) \end{aligned}$$

The challenge ciphertext is constructed as follows:

$$\text{ct} := (\mathbf{sB} + \mathbf{e}_0, \mathbf{s}(\mathbf{B}_1 + \mathbf{C}_{\text{ID}^*}) + \mathbf{e}_1, \mathbf{s}(\mathbf{B}_2 + t^*\mathbf{B}_3) + \mathbf{e}_2, \mathbf{sP} + \mathbf{m}_b \lfloor \frac{q}{2} \rfloor + \mathbf{e}_3)$$

where $\mathbf{s} \leftarrow \mathbb{Z}_q^n$, $\text{ID}^* := (1, \text{id}^*, \dots, (\text{id}^*)^B) \otimes \mathbf{g}'$, $\mathbf{C}_{\text{ID}^*} \leftarrow \text{Com}(\text{pp}, \text{ID}^*)$. The secret key sk for identity set S and batch label t as constructed as follows:

$$\text{sk} \leftarrow \text{SamplePre}([\mathbf{B}|\text{dig}_S|\mathbf{B}_2 + t\mathbf{B}_3], \mathbf{T}_{[\mathbf{B}|\text{dig}_S|\mathbf{B}_2 + t\mathbf{B}_3]}, \mathbf{P}, \sigma_1)$$

where $k = \lceil \log q \rceil$ and

$$\begin{aligned} \mathbf{T}_{[\mathbf{B}|\text{dig}_S|\mathbf{B}_2 + t\mathbf{B}_3]} &\leftarrow \text{ExtendRight}(\mathbf{B}, \mathbf{T}_\mathbf{B}, [\text{dig}_S|\mathbf{B}_2 + t\mathbf{B}_3]), \\ \mathbf{y} &\leftarrow \text{GenCoeff}(S), \\ \text{dig}_S &:= -\mathbf{B}_1 \mathbf{V}(\mathbf{y}^\top \otimes \mathbf{I}_m), \\ \mathbf{V} &\leftarrow \text{Ver}(\text{pp}, 1^{k(B+1)}) \end{aligned}$$

- **Game₀**: This game is the same as **Game_{real}** except that

$$\begin{aligned} (\mathbf{B}_3, \mathbf{T}_{\mathbf{B}_3}) &\leftarrow \text{TrapGen}(1^n, 1^m, q), \\ \mathbf{B}_1 &:= \mathbf{B}\mathbf{U}_1 - \mathbf{C}_{\text{ID}^*}, \\ \mathbf{B}_2 &:= \mathbf{B}\mathbf{U}_2 - t^* \mathbf{B}_3, \\ \mathbf{P} &:= \mathbf{B}\mathbf{U} \end{aligned}$$

where $\mathbf{U}_1, \mathbf{U}_2 \leftarrow \{0, 1\}^{m \times m}$, $\mathbf{U} \leftarrow \{0, 1\}^{m \times \lambda}$. $\mathcal{C}$ keeps $\mathbf{T}_{\mathbf{B}_3}$ in secret.

- **Game₁**: This game is the same as **Game₀** except that

$$\begin{aligned} \mathbf{c}_1 &:= \text{ReRand}(\mathbf{U}_1, \mathbf{c}_0, \chi'/(2\chi)), \\ \mathbf{c}_2 &:= \text{ReRand}(\mathbf{U}_2, \mathbf{c}_0, \chi'/(2\chi)), \\ \mathbf{c}_3 &:= \text{ReRand}(\mathbf{U}, \mathbf{c}_0, \chi'/(2\chi)) + \mathbf{m}_b \lfloor \frac{q}{2} \rfloor \end{aligned}$$

- **Game₂**: This game is the same as **Game₁** except that the way sk for identity set S and batched label $t = t^*$ is generated, where $\text{id}^* \notin S$. To generate such sk , $\mathcal{C}$ computes

$$\text{sk} \leftarrow \text{SamplePre}([\mathbf{B}|\text{dig}_S|\mathbf{B}_2 + t\mathbf{B}_3], \begin{pmatrix} (\mathbf{Z}_{\text{ID}^*} + \mathbf{U}_1 \mathbf{V})(\mathbf{y}^\top \otimes \mathbf{I}_m) \\ \mathbf{I}_m \\ \mathbf{0}^{m \times m} \end{pmatrix}, \mathbf{P}, \sigma_1)$$

- **Game₃**: This game is the same as **Game₂** except that the way sk for identity set S and batched label $t \neq t^*$ generated, where $\text{id}^* \in S$. To generate such sk , $\mathcal{C}$ will do as follows:

- Generate trapdoor by

$$\mathbf{T}' \leftarrow \text{ExtendLeft}([\mathbf{B}|\text{dig}_S], (t - t^*)\mathbf{B}_3, \mathbf{T}_{\mathbf{B}_3}, [\mathbf{U}_2^\top | \mathbf{0}^{m \times m}]^\top)$$

- Compute $\text{sk} \leftarrow \text{SamplePre}([\mathbf{B}|\text{dig}_S|\mathbf{B}_2 + t\mathbf{B}_3], \mathbf{T}', \mathbf{P}, \sigma_1)$

- **Game₄**: This game is the same as **Game₃** except that $\mathbf{c}_0 \leftarrow \mathbb{Z}_q^{1 \times m}$.

- **Game₅**: This game is the same as **Game₄** except that $\mathbf{c}_3 \leftarrow \mathbb{Z}_q^{1 \times \lambda}$.

Lemma 6. Suppose $m > 2n \log q$, we have $\text{Game}_{\text{real}} \approx_s \text{Game}_0$.

Proof. This follows from Left-over Hash Lemma and the properties of **TrapGen** function. $\square$

Lemma 7. Suppose $\chi > \max\{\omega(\sqrt{m}), \omega(\sqrt{\lambda})\}$, $\chi' > 2\chi \max\{m, \lambda\}$, we have $\text{Game}_0 \approx_s \text{Game}_1$

Proof. This follows from the properties of the **ReRand** function, as

$$\begin{aligned} \chi &> \max\{\omega(\sqrt{m}), \omega(\sqrt{\lambda})\}, \\ \chi'/(2\chi) &> \max\{\|\mathbf{U}_1\|, \|\mathbf{U}_2\|, \|\mathbf{U}\|\} \end{aligned}$$

where $\mathbf{c}_0 = \mathbf{s}\mathbf{B} + \mathbf{e}_0$, $\mathbf{e}_0 \leftarrow \mathcal{D}_{\mathbb{Z}^{1 \times m}, \chi}$. $\square$

Lemma 8. Suppose $\sigma_1 > \sigma_0 B \log B \log^2 q \cdot \text{poly}(m, \lambda)$, we have $\text{Game}_1 \approx_s \text{Game}_2$.

Proof. Since $\text{id}^* \notin S$, we have that

$$[\mathbf{B}|\mathbf{B}\mathbf{U}_1] \begin{pmatrix} -\mathbf{Z}_{\text{ID}^*} \\ -\mathbf{V} \end{pmatrix} (\mathbf{y}^\top \otimes \mathbf{I}_m) = (-\mathbf{B}_1 \mathbf{V} - \text{ID}^* \otimes \mathbf{G})(\mathbf{y}^\top \otimes \mathbf{I}_m) = -\text{dig}_S - \text{ID}^* \mathbf{y}^\top \mathbf{G}$$

which means that

$$[\mathbf{B}|\text{dig}_S|\mathbf{B}_2 + t\mathbf{B}_3] \begin{pmatrix} (\mathbf{Z}_{\text{ID}^*} + \mathbf{U}_1 \mathbf{V})(\mathbf{y}^\top \otimes \mathbf{I}_m) \\ \mathbf{I}_m \\ \mathbf{0}^{m \times m} \end{pmatrix} = \text{ID}^* \mathbf{y}^\top \mathbf{G}$$

where $\text{ID}^* \mathbf{y}^\top \neq 0$, $\mathbf{y} = \text{GenCoeff}(S)$. Moreover, trapdoor sampling requires that $\sigma_1 > \sigma_0 B \log B \log^2 q \cdot \text{poly}(m, \lambda)$ because

$$\left\| \begin{pmatrix} (\mathbf{Z}_{\text{ID}^*} + \mathbf{U}_1 \mathbf{V})(\mathbf{y}^\top \otimes \mathbf{I}_m) \\ \mathbf{I}_m \\ \mathbf{0}^{m \times m} \end{pmatrix} \right\| = \sigma_0 B \log B \log^2 q \cdot \text{poly}(m, \lambda)$$

□

Lemma 9. Suppose $\sigma_1 > \omega(\sqrt{n \log q})$, we have $\text{Game}_2 \approx_s \text{Game}_3$.

Proof. Since $t \neq t^*$, we have

$$[\mathbf{B}|\mathbf{B}_2 + t\mathbf{B}_3] = [\mathbf{B}|\mathbf{B}\mathbf{U}_2 + (t - t^*)\mathbf{B}_3]$$

So $\mathcal{C}$ can use $\mathbf{T}_{\mathbf{B}_3}$ to generate the trapdoor for $[\mathbf{B}|\mathbf{B}_2 + t\mathbf{B}_3]$. □

Lemma 10. Under $2m^2$ -succinct LWE assumption, we have $\text{Game}_3 \approx_c \text{Game}_4$.

Proof. Suppose there exists a bit $b \in \{0, 1\}$ and an efficient adversary $\mathcal{A}$ that can distinguish between Game_3 and Game_4 with non-negligible advantage $\epsilon > 0$. We construct an algorithm $\mathcal{C}$ that uses $\mathcal{A}$ as a subroutine to solve the $2m^2$ -succinct LWE problem with non-negligible advantage:

Init: The adversary $\mathcal{A}$ submits a target identity id^* and a target batched label t^* .

Setup: $\mathcal{C}$ uses $2m^2$ to get $(\mathbf{B}, \mathbf{u}, \mathbf{W}, \mathbf{T})$ as an instance of the $2m^2$ -succinct LWE problem. $\mathcal{C}$ samples $\mathbf{U}_1, \mathbf{U}_2 \leftarrow \{0, 1\}^{m \times m}$, $\mathbf{U} \leftarrow \{0, 1\}^{m \times \lambda}$ and $(\mathbf{B}_3, \mathbf{T}_{\mathbf{B}_3}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$. Then, $\mathcal{C}$ sets $\text{ID}^* := (1, \text{id}^*, (\text{id}^*)^2, \dots, (\text{id}^*)^B) \otimes \mathbf{g}'$. Finally, $\mathcal{C}$ gives mpk to $\mathcal{A}$ where

$$\text{mpk} := (\overbrace{\mathbf{B}, \mathbf{W}, \mathbf{T}}^{:=\text{pp}}, \mathbf{B}_1 := \mathbf{B}\mathbf{U}_1 - \mathbf{C}_{\text{ID}^*}, \mathbf{B}_2 := \mathbf{B}\mathbf{U}_2 - t^* \mathbf{B}_3, \mathbf{P} := \mathbf{B}\mathbf{U})$$

among which $\mathbf{C}_{\text{ID}^*} := \text{Com}(\text{pp}, \text{ID}^*)$.

Query Phase 1: The adversary $\mathcal{A}$ may adaptively make secret key queries for identity set S and batched label t which satisfy $\text{id} \notin S$ or $t \neq t^*$:

- If $\text{id}^* \notin S$, $\mathcal{C}$ computes

$$\text{sk} \leftarrow \text{SamplePre}([\mathbf{B}|\text{dig}_S|\mathbf{B}_2 + t\mathbf{B}_3], \begin{pmatrix} (\mathbf{Z}_{\text{ID}^*} + \mathbf{U}_1 \mathbf{V})(\mathbf{y}^\top \otimes \mathbf{I}_m) \\ \mathbf{I}_m \\ \mathbf{0}^{m \times m} \end{pmatrix}, \mathbf{P}, \sigma_1)$$

where

$$\mathbb{Z}_{\text{ID}^*} := \text{Open}(\text{pp}, \text{ID}^*), \mathbf{V} \leftarrow \text{Ver}(\text{pp}, 1^{k(B+1)}), \text{dig}_S \leftarrow \text{Digest}(\text{mpk}, S)$$

- If $\text{id}^* \in S$, $\mathcal{C}$ will do as follows:

- Generate trapdoor by

$$\mathbf{T}' \leftarrow \text{ExtendLeft}([\mathbf{B}|\text{dig}_S], (t - t^*)\mathbf{B}_3, \mathbf{T}_{\mathbf{B}_3}, [\mathbf{U}_2^\top | \mathbf{0}^{m \times m}]^\top)$$

- Compute $\text{sk} \leftarrow \text{SamplePre}([\mathbf{B}|\text{dig}_S|\mathbf{B}_2 + t\mathbf{B}_3], \mathbf{T}', \mathbf{P}, \sigma_1)$

Challenge: The adversary $\mathcal{A}$ submits a pair of challenge messages $(\mathbf{m}_0, \mathbf{m}_1)$. $\mathcal{C}$ outputs challenge ciphertext ct as

$$\text{ct} := \left\{ \begin{array}{ll} \mathbf{c}_0 := \mathbf{u}, & \mathbf{c}_1 := \text{ReRand}(\mathbf{U}_1, \mathbf{u}, \chi'/(2\chi)), \\ \mathbf{c}_2 := \text{ReRand}(\mathbf{U}_2, \mathbf{u}, \chi'/(2\chi)), & \mathbf{c}_3 := \text{ReRand}(\mathbf{U}, \mathbf{u}, \chi'/(2\chi)) + \mathbf{m}_b \lfloor \frac{q}{2} \rfloor \end{array} \right\}$$

Query Phase 2: $\mathcal{A}$ may continue to request a polynomial number of queries like those in Query Phase 1, $\mathcal{C}$ replies $\mathcal{A}$ as in Query Phase 1.

Guess: The adversary $\mathcal{A}$ outputs a guess b' and $\mathcal{C}$ outputs $b = b'$.

It is clear that $\mathcal{C}$ correctly simulates an execution of Game_3 and Game_4 for $\mathcal{A}$:

- The way for selecting public parameters is the same as in Game_3 and Game_4 .
- The way for generating sk is the same as in Game_3 and Game_4 .
- Consider the challenge ciphertext:
 - If $\mathbf{u} = \mathbf{sB} + \mathbf{e}_0$, then $\mathbf{c}_0 = \mathbf{sB} + \mathbf{e}_0$ which is the same as in Game_3 .
 - If $\mathbf{u} \leftarrow \mathbb{Z}_q^{1 \times m}$, then $\mathbf{c}_0 = \mathbf{u}$ assumes a uniform distribution on $\mathbb{Z}_q^{1 \times m}$ which is the same as in Game_4 .

□

Lemma 11. Suppose $m > 2(n + 1) \log q$, we have $\text{Game}_4 \approx_s \text{Game}_5$.

Proof. This follows from Left-over Hash Lemma. □

□

Acknowledgements. We are grateful to Hoeteck Wee for his insightful advice and helpful discussions.

References

- [ABB10] Shweta Agrawal, Dan Boneh, and Xavier Boyen. Efficient lattice (H)IBE in the standard model. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 553–572. Springer, Berlin, Heidelberg, May / June 2010. [doi:10.1007/978-3-642-13190-5_28](https://doi.org/10.1007/978-3-642-13190-5_28).
- [AFP25] Amit Agarwal, Rex Fernando, and Benny Pinkas. Efficiently-thresholdizable batched identity based encryption, with applications. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 69–100. Springer, Cham, August 2025. [doi:10.1007/978-3-032-01881-6_3](https://doi.org/10.1007/978-3-032-01881-6_3).

- [AFV11] Shweta Agrawal, David Mandell Freeman, and Vinod Vaikuntanathan. Functional encryption for inner product predicates from learning with errors. In Dong Hoon Lee and Xiaoyun Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 21–40. Springer, Berlin, Heidelberg, December 2011. doi:10.1007/978-3-642-25385-0_2.
- [BFOQ24] Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyang Qu. BEAT-MEV: Epochless approach to batched threshold encryption for MEV prevention. Cryptology ePrint Archive, Report 2024/1533, 2024. URL: <https://eprint.iacr.org/2024/1533>.
- [BGG⁺14] Dan Boneh, Craig Gentry, Sergey Gorbunov, Shai Halevi, Valeria Nikolaenko, Gil Segev, Vinod Vaikuntanathan, and Dhinakaran Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit ABE and compact garbled circuits. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 533–556. Springer, Berlin, Heidelberg, May 2014. doi:10.1007/978-3-642-55220-5_30.
- [BJK15] Allison Bishop, Abhishek Jain, and Lucas Kowalczyk. Function-hiding inner product encryption. In Tetsu Iwata and Jung Hee Cheon, editors, *ASIACRYPT 2015, Part I*, volume 9452 of *LNCS*, pages 470–491. Springer, Berlin, Heidelberg, November / December 2015. doi:10.1007/978-3-662-48797-6_20.
- [BKP13] Rikke Bendlin, Sara Krehbiel, and Chris Peikert. How to share a lattice trapdoor: Threshold protocols for signatures and (H)IBE. In Michael J. Jacobson, Jr., Michael E. Locasto, Payman Mohassel, and Reihaneh Safavi-Naini, editors, *ACNS 2013*, volume 7954 of *LNCS*, pages 218–236. Springer, Berlin, Heidelberg, June 2013. doi:10.1007/978-3-642-38980-1_14.
- [BLT25] Dan Boneh, Evan Laufer, and Ertem Nusret Tas. Batch decryption without epochs and its application to encrypted mempools. *Cryptology ePrint Archive*, 2025. URL: <https://eprint.iacr.org/2025/1254>.
- [CGPP24] Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. Mempool privacy via batched threshold encryption: Attacks and defenses. In Davide Balzarotti and Wenyan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/choudhuri>.
- [CGPW24] Arka Rai Choudhuri, Sanjam Garg, Guru-Vamsi Policharla, and Mingyuan Wang. Practical mempool privacy via one-time setup batched threshold encryption. *Cryptology ePrint Archive*, Report 2024/1516, 2024. URL: <https://eprint.iacr.org/2024/1516>.
- [CHW25] Jeffrey Champion, Yao-Ching Hsieh, and David J. Wu. Registered ABE and adaptively-secure broadcast encryption from succinct LWE. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 3–34. Springer, Cham, August 2025. doi:10.1007/978-3-032-01881-6_1.
- [CLM23] Valerio Cini, Russell W. F. Lai, and Giulio Malavolta. Lattice-based succinct arguments from vanishing polynomials - (extended abstract). In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 72–105. Springer, Cham, August 2023. doi:10.1007/978-3-031-38545-2_3.

- [CLR16] Jie Chen, Benoît Libert, and Somindu C. Ramanna. Non-zero inner product encryption with short ciphertexts and private keys. In Vassilis Zikas and Roberto De Prisco, editors, *SCN 16*, volume 9841 of *LNCS*, pages 23–41. Springer, Cham, August / September 2016. doi:10.1007/978-3-319-44618-9_2.
- [CW24] Jeffrey Champion and David J. Wu. Distributed broadcast encryption from lattices. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part III*, volume 15366 of *LNCS*, pages 156–189. Springer, Cham, December 2024. doi:10.1007/978-3-031-78020-2_6.
- [GWW25] Junqing Gong, Brent Waters, Hoeteck Wee, and David J. Wu. Threshold batched identity-based encryption from pairings in the plain model. Cryptology ePrint Archive, Paper 2025/2103, 2025. URL: <https://eprint.iacr.org/2025/2103>.
- [KLM⁺18] Sam Kim, Kevin Lewi, Avradip Mandal, Hart Montgomery, Arnab Roy, and David J. Wu. Function-hiding inner product encryption is practical. In Dario Catalano and Roberto De Prisco, editors, *SCN 18*, volume 11035 of *LNCS*, pages 544–562. Springer, Cham, September 2018. doi:10.1007/978-3-319-98113-0_29.
- [KNYY20] Shuichi Katsumata, Ryo Nishimaki, Shota Yamada, and Takashi Yamakawa. Adaptively secure inner product encryption from LWE. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part III*, volume 12493 of *LNCS*, pages 375–404. Springer, Cham, December 2020. doi:10.1007/978-3-030-64840-4_13.
- [KSW08] Jonathan Katz, Amit Sahai, and Brent Waters. Predicate encryption supporting disjunctions, polynomial equations, and inner products. In Nigel P. Smart, editor, *EUROCRYPT 2008*, volume 4965 of *LNCS*, pages 146–162. Springer, Berlin, Heidelberg, April 2008. doi:10.1007/978-3-540-78967-3_9.
- [KY16] Shuichi Katsumata and Shota Yamada. Partitioning via non-linear polynomial functions: More compact IBEs from ideal lattices and bilinear maps. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 682–712. Springer, Berlin, Heidelberg, December 2016. doi:10.1007/978-3-662-53890-6_23.
- [MP12] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 700–718. Springer, Berlin, Heidelberg, April 2012. doi:10.1007/978-3-642-29011-4_41.
- [Sha84] Adi Shamir. Identity-based cryptosystems and signature schemes. In G. R. Blakley and David Chaum, editors, *CRYPTO’84*, volume 196 of *LNCS*, pages 47–53. Springer, Berlin, Heidelberg, August 1984. doi:10.1007/3-540-39568-7_5.
- [Sho97] Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *EUROCRYPT’97*, volume 1233 of *LNCS*, pages 256–266. Springer, Berlin, Heidelberg, May 1997. doi:10.1007/3-540-69053-0_18.
- [Wee24] Hoeteck Wee. Circuit ABE with $\text{poly}(\text{depth}, \lambda)$ -sized ciphertexts and keys from lattices. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part III*, volume 14922 of *LNCS*, pages 178–209. Springer, Cham, August 2024. doi:10.1007/978-3-031-68382-4_6.

- [Wee25] Hoeteck Wee. Almost optimal KP and CP-ABE for circuits from succinct LWE. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part III*, volume 15603 of *LNCS*, pages 34–62. Springer, Cham, May 2025. [doi:10.1007/978-3-031-91131-6_2](https://doi.org/10.1007/978-3-031-91131-6_2).
- [WFW17] Zhedong Wang, Xiong Fan, and Mingsheng Wang. Compact inner product encryption from LWE. In Sihan Qing, Chris Mitchell, Liqun Chen, and Dongmei Liu, editors, *ICICS 17*, volume 10631 of *LNCS*, pages 141–153. Springer, Cham, December 2017. [doi:10.1007/978-3-319-89500-0_12](https://doi.org/10.1007/978-3-319-89500-0_12).
- [WW23a] Hoeteck Wee and David J. Wu. Lattice-based functional commitments: Fast verification and cryptanalysis. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part V*, volume 14442 of *LNCS*, pages 201–235. Springer, Singapore, December 2023. [doi:10.1007/978-981-99-8733-7_7](https://doi.org/10.1007/978-981-99-8733-7_7).
- [WW23b] Hoeteck Wee and David J. Wu. Succinct vector, polynomial, and functional commitments from lattices. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part III*, volume 14006 of *LNCS*, pages 385–416. Springer, Cham, April 2023. [doi:10.1007/978-3-031-30620-4_13](https://doi.org/10.1007/978-3-031-30620-4_13).
- [WW25] Hoeteck Wee and David J. Wu. Unbounded distributed broadcast encryption and registered ABE from succinct LWE. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 204–235. Springer, Cham, August 2025. [doi:10.1007/978-3-032-01881-6_7](https://doi.org/10.1007/978-3-032-01881-6_7).
- [Xag13] Keita Xagawa. Improved (hierarchical) inner-product encryption from lattices. In Kaoru Kurosawa and Goichiro Hanaoka, editors, *PKC 2013*, volume 7778 of *LNCS*, pages 235–252. Springer, Berlin, Heidelberg, February / March 2013. [doi:10.1007/978-3-642-36362-7_15](https://doi.org/10.1007/978-3-642-36362-7_15).